6

ПРОЕКТИРОВАНИЕ ХРАНИЛИЩА ТИПА «КЛЮЧ–ЗНАЧЕНИЕ»

Хранилище типа «ключ–значение» — это нереляционная база данных. Каждый уникальный идентификатор хранится в виде ключа и имеет отдельное значение.

В паре «ключ–значение» ключ должен быть уникальным, а соответствующее значение может быть получено через ключ. Ключами могут выступать как обычный текст, так и хешированные значения. Короткие ключи обеспечивают лучшую производительность. Как они выглядят? Вот несколько примеров:

- ключ в виде обычного текста: last_logged_in_at;
- хешированный ключ: 253DDEC4.

Значение в паре может быть строкой, списком, объектом и т. д. В таких хранилищах, как Amazon Dynamo [1], Memcached [2], Redis [3] и т. д., значения обычно считают непрозрачными объектами.

Вот фрагмент данных из хранилища типа «ключ–значение»:

Таблица 6.1

ключ	значение
145	john
147	bob
160	julia

В этой главе вам предложено спроектировать хранилище типа «ключ–значение» с поддержкой следующих операций:

- put(ключ, значение) // вставить «значение», связанное с «ключом»;
- get(ключ) // получить «значение», связанное с «ключом».

ПОНЯТЬ ЗАДАЧУ И ОПРЕДЕЛИТЬ МАСШТАБ РЕШЕНИЯ

Идеальной архитектуры не существует. Каждое решение подразумевает определенные компромиссы между чтением, записью и расходом памяти. Двумя другими факторами, между которыми нужно найти баланс, являются согласованность и доступность. В этой главе мы спроектируем хранилище типа «ключ–значение», обладающее следующими характеристиками:

- малый размер пар «ключ–значение»: меньше 10 Кб;
- возможность хранения объемных данных;
- высокая доступность — система должна быстро отвечать даже во время сбоев;
- хорошая масштабируемость — система должна масштабироваться для поддержки объемных наборов данных;
- автоматическое масштабирование — серверы должны автоматически добавляться/удаляться в зависимости от трафика;
- регулируемая согласованность;
- низкая латентность.

ХРАНИЛИЩЕ ТИПА «КЛЮЧ–ЗНАЧЕНИЕ» НА ОДНОМ СЕРВЕРЕ

Разработать хранилище типа «ключ–значение», находящееся в пределах одного сервера, довольно просто. Очевидное решение заключается в хранении пар в хеш-таблице, содержимое которой находится в памяти. Достать данные оттуда довольно легко, но их объем ограничен из-за лимита памяти. С этой проблемой можно бороться двумя путями:

- сжимать данные;
- хранить в памяти только часто используемые данные, а все остальное записывать на диск.

Но даже с этими мерами отдельно взятый сервер может очень быстро исчерпать свои ресурсы. Для поддержки крупных данных хранилище типа «ключ–значение» должно быть распределенным.

РАСПРЕДЕЛЕННОЕ ХРАНИЛИЩЕ ТИПА «КЛЮЧ–ЗНАЧЕНИЕ»

Распределенное хранилище типа «ключ–значение» иногда называют распределенной хеш-таблицей. Оно распределяет пары «ключ–значение» между множеством серверов. При проектировании распределенной системы необходимо понимать теорему CAP (Consistency, Availability, Partition Tolerance — «согласованность, доступность, устойчивость к секционированию»).

Теорема CAP

Теорема CAP гласит, что распределенная система может обеспечивать не больше двух из следующих трех свойств: согласованность, доступность и устойчивость к секционированию. Дадим несколько определений.

- **Согласованность.** Означает, что все клиенты одновременно видят одни и те же данные, к какому бы узлу они ни подключились.
- **Доступность.** Означает, что любой клиент, запрашивающий данные, получает ответ, даже если некоторые из узлов недоступны.
- **Устойчивость к секционированию.** Секционирование свидетельствует о нарушении связи между двумя узлами. Устойчивость к секционированию означает, что система продолжает работать вопреки нарушению связи в сети.

Согласно теореме CAP, одним из этих свойств необходимо пожертвовать, чтобы обеспечить поддержку двух других (рис. 6.1).

Сейчас хранилища типа «ключ–значение» классифицируются в зависимости от того, какие две характеристики CAP они поддерживают:

- **Системы CP (согласованность и устойчивость к секционированию).** Жертвуют доступностью.
- **Системы AP (доступность и устойчивость к секционированию).** Жертвуют согласованностью.
- **Системы CA (согласованность и доступность).** Жертвуют устойчивостью к секционированию. Поскольку сетевые сбои неизбежны, распределенные системы должны справляться с разделением сети. В связи с этим системы CA не существуют в реальных условиях.



Рис. 6.1

Все это лишь определения. Чтобы вам было легче понять, о чем идет речь, рассмотрим несколько конкретных примеров. В распределенных системах данные обычно реплицируются больше одного раза. Предположим, что у нас есть три узла-реплики: n1, n2 и n3, как показано на рис. 6.2.

Идеальная ситуация

В идеальном мире секционирование сети не происходит. Данные, записанные в n1, автоматически реплицируются в n2 и n3. Этим достигается как согласованность, так и доступность.

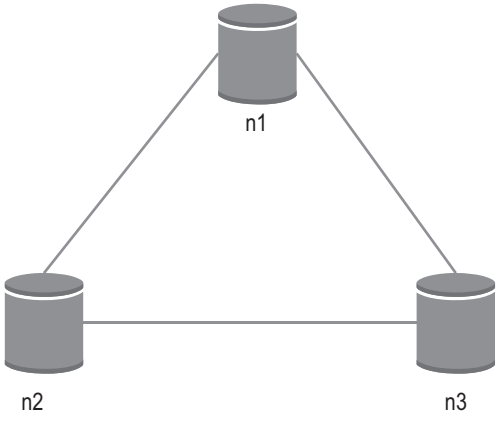


Рис. 6.2

Реальные распределенные системы

В распределенной системе разделение неизбежно, и, когда оно происходит, мы должны сделать выбор между согласованностью и доступностью. На рис. 6.3 узел n3 отказывает и больше не может взаимодействовать с n1 и n2. Данные, которые клиенты записывают в n1 или n2, не могут дойти до n3. Если же кто-то запишет данные в n3 и они не успеют дойти до n1 и n2, это будет означать, что содержимое n1 и n2 неактуально.

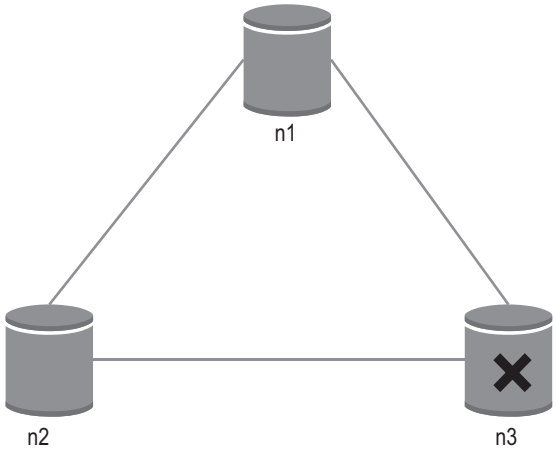


Рис. 6.3

Если мы предпочтем согласованность вместо доступности (система CP), нам придется заблокировать все операции записи на узлах n1 и n2, чтобы избежать рассинхронизации данных между этими тремя серверами. Этим мы сделаем систему недоступной. Чрезвычайно высокие требования к согласованности обычно имеют банковские системы. Например, для банка крайне важно отобразить самую актуальную информацию о балансе клиента. Если из-за разделения сети произойдет рассинхронизация, банковская система начнет возвращать ошибки, пока проблема не будет устранена.

Если же мы отдадим предпочтение доступности перед согласованностью (система AP), система продолжит принимать операции чтения, несмотря на то что она может вернуть устаревшие данные. Что касается операций записи, то они останутся доступными на узлах n1 и n2, а после устранения сетевых неполадок данные будут синхронизированы с n3.

Выбор правильных CAP, которые подходят для вашей задачи, — важный этап создания распределенного хранилища типа «ключ–значение». Вы можете обсудить это со своим интервьюером и спроектировать систему соответствующим образом.

Компоненты системы

В этом разделе мы обсудим основные компоненты и методики, которые используются для создания хранилища типа «ключ–значение»:

- секционирование данных;
- репликация данных;
- согласованность;
- устранение несогласованности;
- обработка сбоев;
- диаграмма архитектуры системы;
- маршрут записи;
- маршрут чтения.

Следующий материал во многом основан на трех популярных системах хранения данных типа «ключ–значение»: Dynamo [4], Cassandra [5] и BigTable [6].

Секционирование данных

В крупных приложениях все данные не могут поместиться на одном сервере. Проще всего было бы разделить их на части (секции) меньшего размера и хранить на разных серверах. При секционировании данных необходимо решить две проблемы:

- равномерно распределить их между разными серверами;
- минимизировать их перемещение при добавлении или удалении узлов.

В качестве решения отлично подойдет согласованное хеширование, рассмотренное в главе 5. Давайте вспомним в общих чертах, как оно работает.

- Сначала серверы наносятся на кольцо хеширования. На рис. 6.4 показано кольцо, где есть 8 серверов, обозначенных как s0, s1, ..., s7.
- Затем на том же кольце хешируется ключ, который сохраняется на ближайшем сервере по часовой стрелке. Например, по этой логике ключ 0 сохраняется в s1.

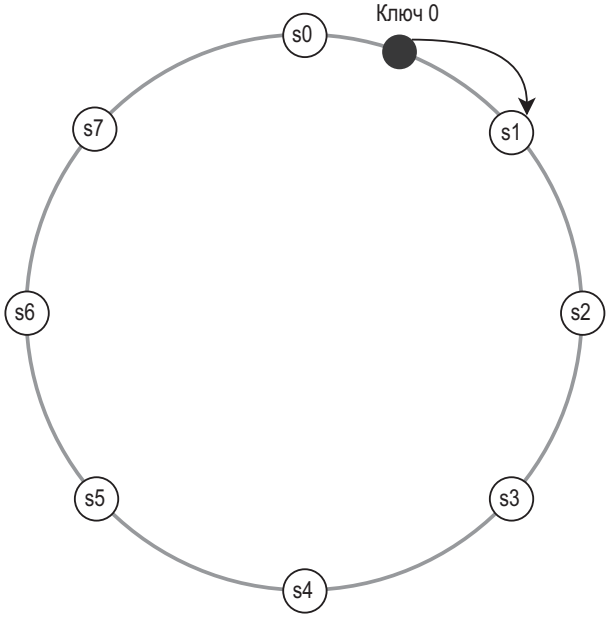


Рис. 6.4

Использование согласованного хеширования для секционирования данных имеет следующие преимущества.

- **Автоматическое масштабирование.** Серверы можно добавлять и удалять автоматически в зависимости от загрузки.
- **Гетерогенность.** Количество виртуальных узлов сервера пропорционально его емкости. Например, серверам с большей емкостью назначают больше виртуальных узлов.

Репликация данных

Чтобы достичь высокой доступности и надежности, данные должны асинхронно реплицироваться по N серверам, причем параметр N можно настраивать. Эти N серверов выбираются по такому принципу: после нанесения ключа на кольцо хеширования мы двигаемся по часовой стрелке от его позиции и выбираем ближайшие N серверов для хранения копий данных. На рис. 6.5 $N = 3$ и ключ 0 реплицируется между серверами s1, s2 и s3.

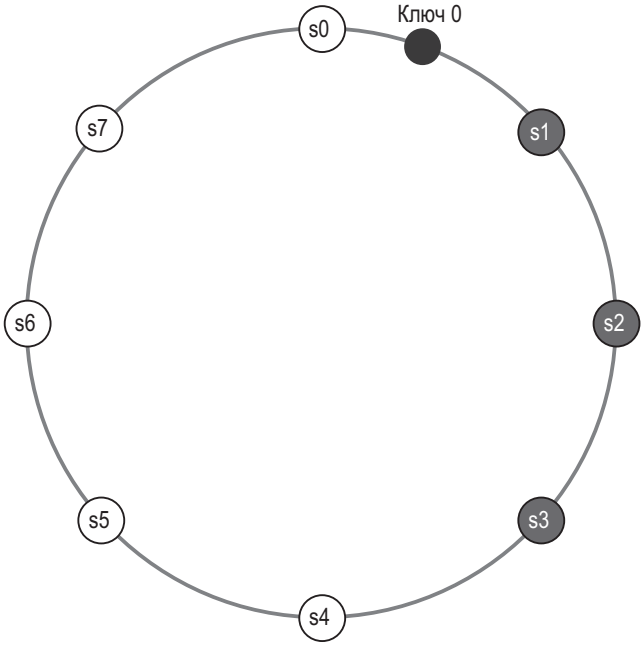


Рис. 6.5

Первые N виртуальных узлов могут принадлежать физическим серверам, количество которых меньше N. Чтобы избежать этой проблемы, при прохождении по часовой стрелке выбираются только уникальные серверы.

Узлы, находящиеся в одном центре обработки данных, зачастую выходят из строя одновременно в результате отключения электричества, неполадок в сети, стихийных бедствий и т. д. Для повышения надежности системы реплики размещаются в разных ЦОД, между которыми установлены высокоскоростные сетевые соединения.

Согласованность

Так как данные реплицируются по нескольким узлам, они должны синхронизироваться между репликами. Консенсус кворума может обеспечить согласованность как чтения, так и записи. Для начала перечислим несколько определений.

- **N** = количество реплик.
- **W** = кворум записи размера W. Операция записи считается успешной, только если она подтверждена W репликами.
- **R** = кворум чтения размера R. Чтобы операцию записи можно было считать успешной, необходимо дождаться ответа как минимум от R реплик.

Рассмотрим пример на рис. 6.6 с $N = 3$.

$W = 1$ не означает, что данные записаны на одном сервере. Например, в конфигурации, представленной на рис. 6.6, данные реплицируются по $s0$, $s1$ и $s2$. Значение $W = 1$ говорит о том, что, прежде чем считать операцию записи успешной, координатор должен получить как минимум одно подтверждение. То есть если сервер $s1$ подтвердит операцию, нам больше не нужно ждать подтверждений от $s0$ и $s2$. Координатор выступает прокси-сервером между клиентом и узлами.

Выбор значений для W, R и N — это типичный компромисс между латентностью и согласованностью. Если $W = 1$ или $R = 1$, операция завершается быстро, так как координатору нужно ждать ответа только от одной из реплик. Если же W или R больше 1, система становится более согласованной, но при этом координатору придется ждать ответа от самой медленной реплики, что замедлит выполнение запросов.

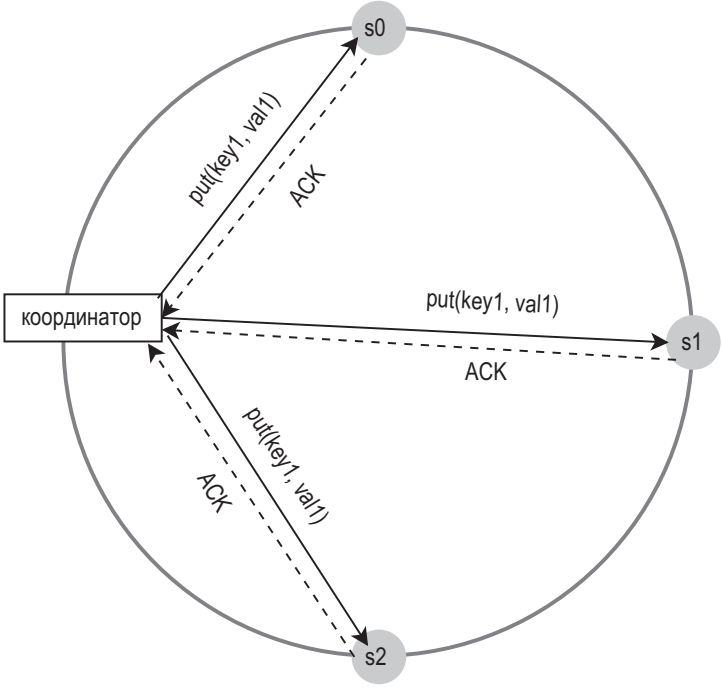


Рис. 6.6 (ACK = квитирование)

$W + R > N$ гарантирует строгую согласованность, поскольку в системе должен быть как минимум один узел с тем же минимальным набором данных.

Как сконфигурировать N, W и R для наших задач? Вот несколько возможных вариантов:

- если $R = 1$ и $W = N$, система оптимизирована для быстрого чтения;
- если $W = 1$ и $R = N$, система оптимизирована для быстрой записи;
- если $W + R > N$, гарантируется строгая согласованность (обычно $N = 3$, $W = R = 2$);
- если $W + R <= N$, строгая согласованность не гарантируется.

В зависимости от требований значения W, R, N можно оптимизировать для получения нужного уровня согласованности.

Модели согласованности

Модель согласованности — еще один важный фактор, который следует учитывать при проектировании хранилища типа «ключ–значение». Она определяет степень согласованности данных и имеет широкий спектр разновидностей.

- Строгая согласованность. Любая операция чтения возвращает значение, соответствующее результату самой последней операции записи. Клиент всегда получает актуальные данные.
- Слабая согласованность. Последующие операции чтения могут и не вернуть самое последнее значение.
- Согласованность в конечном счете. Это разновидность слабой согласованности. Рано или поздно все обновления распространяются по системе и все реплики становятся согласованными.

Жесткая согласованность обычно достигается за счет того, что операции чтения/записи принимаются только после подтверждения текущей записи всеми репликами. Это не самый оптимальный подход для высокодоступных систем, так как он может блокировать новые операции. В Dynamo и Cassandra используется отложенная согласованность, и именно эту модель мы рекомендуем для нашего хранилища. Она допускает поступление в систему несогласованных значений, заставляя клиента их прочитать и согласовать. В следующем разделе мы рассмотрим процесс согласования на основе версионирования.

Устранение несогласованности: версионирование

Репликация обеспечивает высокую доступность, но при этом делает реплики несогласованными. Для решения этой проблемы применяются версионирование и векторные часы. Версионирование — это когда каждое обновление данных приводит к появлению их новой неизменяемой версии. Прежде чем переходить к этой теме, обсудим пример возникновения несогласованности.

Как показано на рис. 6.7, узлы-реплики n1 и n2 имеют одно и то же значение. Назовем его *исходным*. Сервер 1 и сервер 2 получают одно и то же значение при выполнении операции `get("name")`.

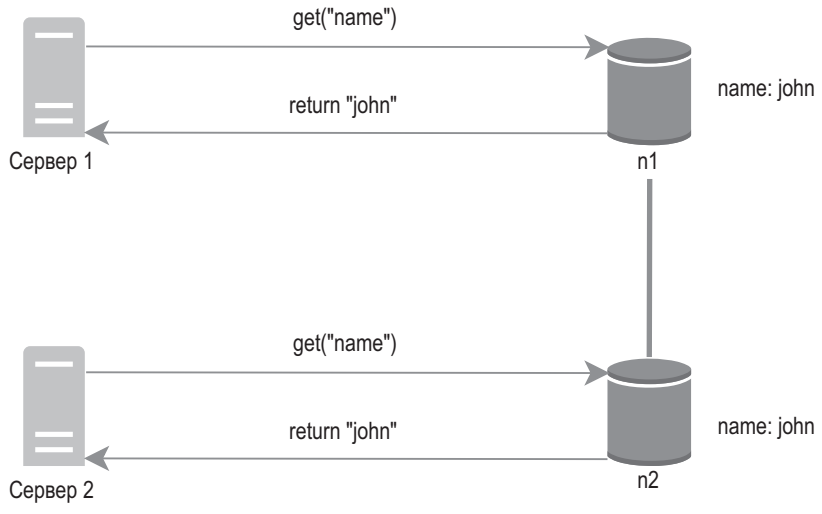


Рис. 6.7

Далее, как видно на рис. 6.8, сервер 1 меняет значение name на johnSanFrancisco, а сервер 2 меняет то же значение на johnNewYork. Эти два изменения производятся одновременно. Мы получаем два конфликтующих значения, которые называются версиями v1 и v2.

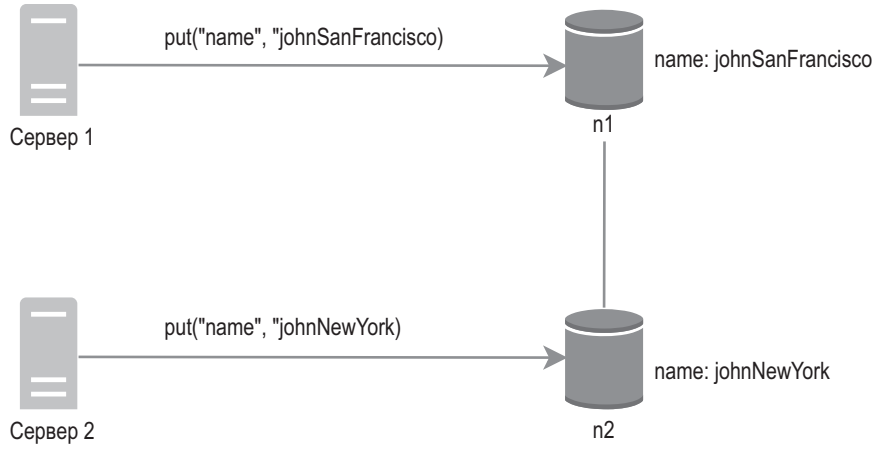


Рис. 6.8

В этом примере исходное значение можно игнорировать, так как на нем были основаны изменения. Однако конфликт между двумя последними версиями нельзя разрешить каким-то очевидным способом. Нам нужна система версионирования, способная обнаруживать и улаживать конфликты. Для решения этой проблемы часто применяют методику, известную как векторные часы. Давайте посмотрим, как она работает.

Векторные часы — это пара [сервер, версия], связанная с элементом данных. С ее помощью можно проверить, какая из двух версий более новая и есть ли между ними конфликт.

Допустим, у нас есть векторные часы вида *D([S1, v1], [S2, v2], ..., [Sn, vn])*, где *D* — элемент данных, *v1* — номер версии, а *s1* — номер сервера. Когда элемент данных *D* записывается на сервер *Si*, система должна выполнить одно из следующих действий:

- инкрементировать *vi*, если *[Si, vi]* существует;
- в противном случае создать новую запись *[Si, vi]*.

Конкретный пример использования этой абстрактной логики показан на рис. 6.9.

1. Клиент записывает в систему элемент данных D1, и запись обрабатывается сервером Sx, у которого теперь есть векторные часы D1[(Sx, 1)].
2. Другой клиент считывает последнюю версию D1, обновляет ее до D2 и записывает обратно. D2 происходит от элемента D1 и поэтому записывается вместо него. Предполагается, что запись обрабатывается тем же сервером Sx, векторные часы которого теперь выглядят как D2([Sx, 2]).
3. Еще один клиент считывает последнюю версию D2, обновляет ее до D3 и записывает обратно. Предполагается, что запись обрабатывается тем же сервером Sy, векторные часы которого теперь выглядят как D3([Sx, 2], [Sy, 1])).
4. Еще один клиент считывает последнюю версию D2, обновляет ее до D4 и записывает обратно. Предполагается, что запись обрабатывается тем же сервером Sz, векторные часы которого теперь выглядят как D4([Sx, 2], [Sz, 1])).

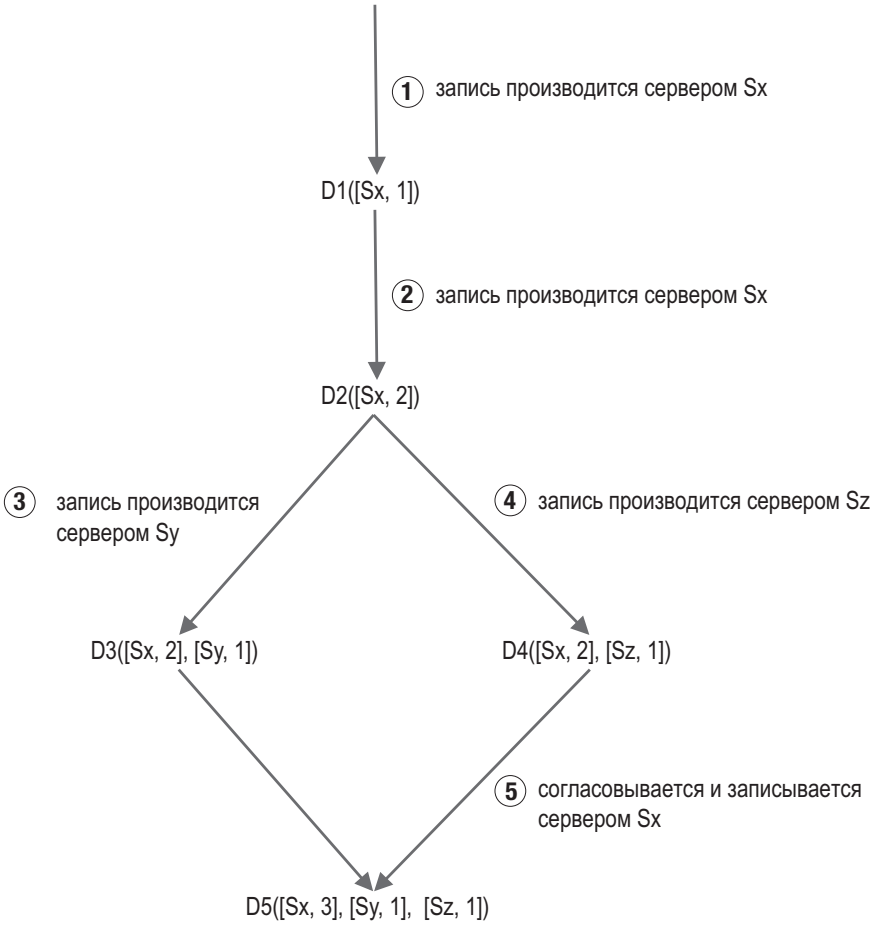


Рис. 6.9

5. Следующий клиент, который считывает D3 и D4, обнаруживает конфликт, вызванный тем, что элемент данных D2 был изменен двумя серверами: Sy и Sz. Клиент разрешает этот конфликт и отправляет на сервер обновленные данные. Предполагается, что запись обрабатывается тем же сервером Sx, который теперь имеет значение D5([Sx, 3], [Sy, 1], [Sz, 1]). О том, как обнаруживать конфликты, мы поговорим чуть позже.

Если номер версии каждого члена векторных часов *Y* больше или равен номерам версий в *X*, это означает, что *X* является наследником *Y* и, сле-

довательно, эти версии не конфликтуют. Например, векторные часы D([s0, 1], [s1, 1])] являются предшественником D([s0, 1], [s1, 2]), поэтому конфликт не записывается.

Точно так же очевидно, что *X* и *Y* находятся на одном уровне (то есть конфликтуют между собой), если любой член векторных часов *Y* имеет версию ниже, чем у соответствующего члена *X*. Например, векторные часы D([s0, 1], [s1, 2]) и D([s0, 2], [s1, 1]) сигнализируют о конфликте.

Векторные часы могут разрешать конфликты, но у них есть два заметных недостатка. Во-первых, они усложняют клиент, так как в нем должна быть реализована логика разрешения конфликтов.

Во-вторых, пары [сервер, версия] в векторных часах могут очень быстро накапливаться. Чтобы это исправить, мы можем указать максимальную длину, при превышении которой самая старая пара удаляется. Это может снизить эффективность процесса согласования, так как больше нельзя будет точно сказать, является ли версия потомком. Но, судя по исследованию [4], компания Amazon еще не сталкивалась с этой проблемой в ходе промышленной эксплуатации Dynamo; следовательно, это решение должно быть приемлемым для большинства организаций.

Обработка сбоев

В любых крупномасштабных системах сбои являются не только неизбежным, но и довольно распространенным явлением. Их обработка имеет большое значение. В этом разделе мы сначала познакомимся с методами обнаружения сбоев, а затем пройдемся по распространенным стратегиям их обработки.

Обнаружение сбоев

В распределенной системе о поломке сервера нельзя судить только по сигналам с другого сервера. Обычно для этого требуется подтверждение от двух независимых источников.

На рис. 6.10 показано мультивещание вида «все ко всем». Это простое и понятное решение, но при большом количестве серверов в системе оно становится неэффективным.

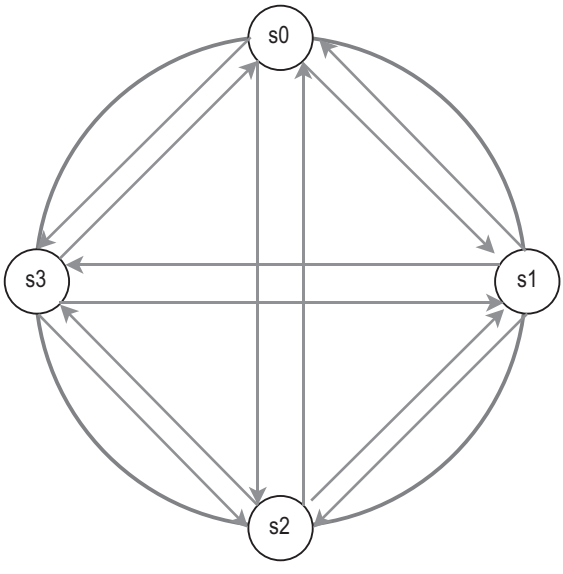


Рис. 6.10

Более оптимальное решение состоит в использовании децентрализованных методов обнаружения сбоев, таких как протокол сплетен (gossip protocol). Он работает следующим образом.

- Каждый узел хранит список узлов-участников, состоящий из идентификаторов и счетчиков пульсации.
- Каждый узел периодически инкрементирует счетчик пульсации.
- Каждый узел периодически шлет пульс группе произвольных узлов, которые в свою очередь передают его другой группе.
- При получении пульса узлы обновляют список участников до последней версии.
- Если счетчик пульсации не увеличивается на протяжении заранее определенного периода, участник считается недоступным.

Как показано на рис. 6.11:

- Узел s0 хранит список участников, показанных слева.
- Узел s0 замечает, что счетчик пульсации узла s2 (ID участника 2) уже давно не увеличивался.

- Узел s0 шлет группе произвольных узлов пульс с информацией об s2. Когда другие узлы подтвердят, что счетчик пульсации s2 давно не обновлялся, узел s2 будет помечен как недоступный и информация об этом будет передана другим узлам.

s0's membership list

ID участника	Счетчик пульсации	Время
0	10232	12:00:01
1	10224	12:00:10
2	**9908**	**11:58:02**
3	10237	12:00:20
4	10234	12:00:34

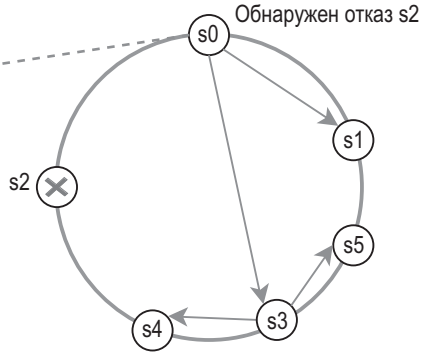


Рис. 6.11

Обработка временных сбоев

После обнаружения сбоя по протоколу сплетен система должна задействовать определенные механизмы, чтобы обеспечить доступность. Если используется строгий кворум, операции чтения и записи могут быть заблокированы, как было проиллюстрировано в разделе о консенсусе кворума.

Для повышения доступности используется методика, известная как нестрогий кворум [4]. Вместо обеспечения кворума система выбирает на кольце хеширования первые W исправных серверов для записи и первые R исправных серверов для чтения. Недоступные серверы игнорируются.

Если один сервер недоступен из-за сетевых неполадок, вместо него обработкой запросов временно займется другой. Когда сеть возобновит нормальную работу, изменения будут переданы ранее недоступному серверу, чтобы восстановить согласованность данных. Этот процесс называют прозрачной передачей. На рис. 6.12 сервер s2 недоступен, поэтому операции чтения и записи будут временно обрабатываться сервером s3. Когда s2 снова появится в сети, s3 передаст ему измененные данные.

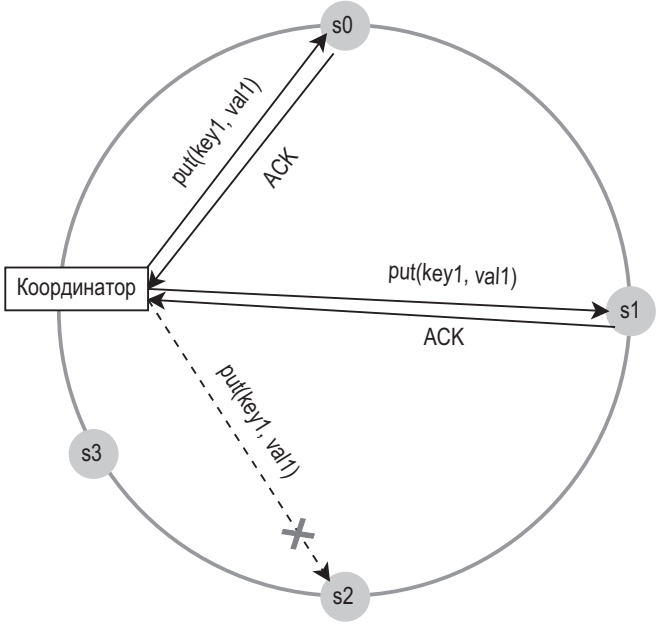


Рис. 6.12

Обработка бессрочных сбоев

Прозрачная передача используется при временных сбоях. Но что, если реплика пропадает безвозвратно? Чтобы справиться с этой ситуацией, нужно реализовать протокол для предотвращения энтропии, который будет синхронизировать реплики. Это подразумевает сравнение элементов данных, хранящихся на репликах, и обновление каждой реплики до самой новой версии. Для обнаружения несогласованности и минимизации объема передаваемых данных используется дерево Меркла.

Цитата из Википедии [7]: «Хеш-деревом, деревом Меркла (Merkle tree), называют полное двоичное дерево, в листовые вершины которого помещены хеши от блоков данных, а внутренние вершины содержат хеши от сложения значений в дочерних вершинах. Хеш-деревья обеспечивают эффективную и безопасную проверку содержимого крупных структур данных».

Если наши ключи находятся в диапазоне от 1 до 12, построить дерево Меркла можно с помощью нескольких шагов (выделенные ячейки обозначают несогласованность).

Шаг 1: разделить диапазон ключей на бакеты (в нашем примере их 4), как показано на рис. 6.13. Бакет выступает корневым узлом, что позволяет ограничить глубину дерева.

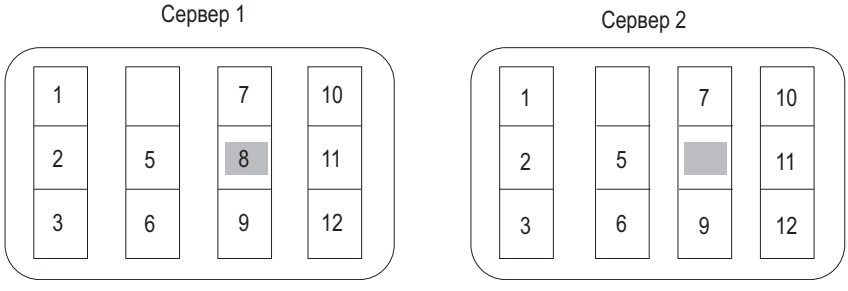


Рис. 6.13

Шаг 2: после создания бакетов захешировать для каждого из них ключ с помощью одного и того же метода хеширования (рис. 6.14).

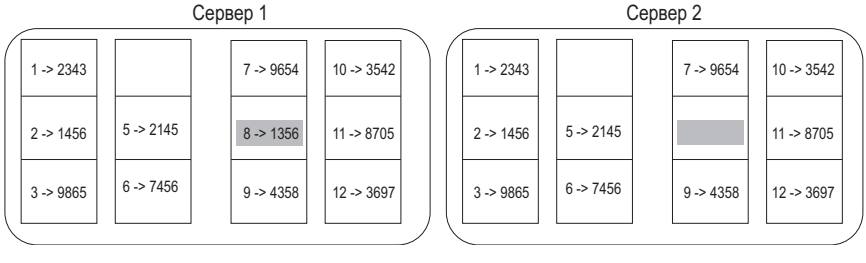


Рис. 6.14

Шаг 3: создать по одному хеш-узлу для каждого бакета (рис. 6.15).

Шаг 4: построить дерево снизу вверх до самого корня путем вычисления дочерних хешей (рис. 6.16).

Сравнение двух деревьев Меркла начинается с их корневых хешей. Если корневые хеши совпадают, серверы содержат одни и те же данные. В противном случае сравниваются дочерние хеши слева направо. В процессе осмотра дерева мы определяем несогласованные бакеты и синхронизируем только их.

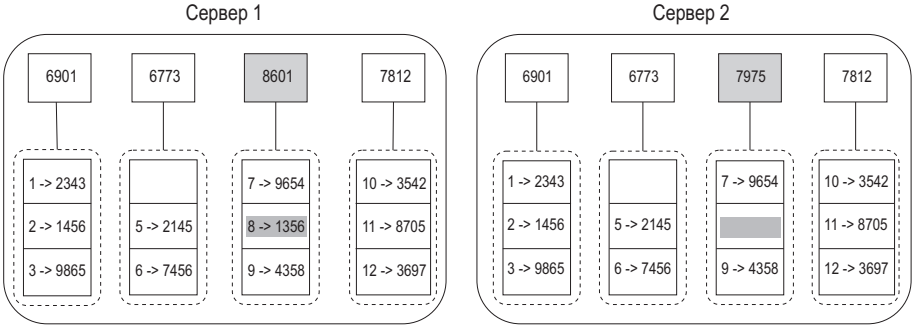


Рис. 6.15

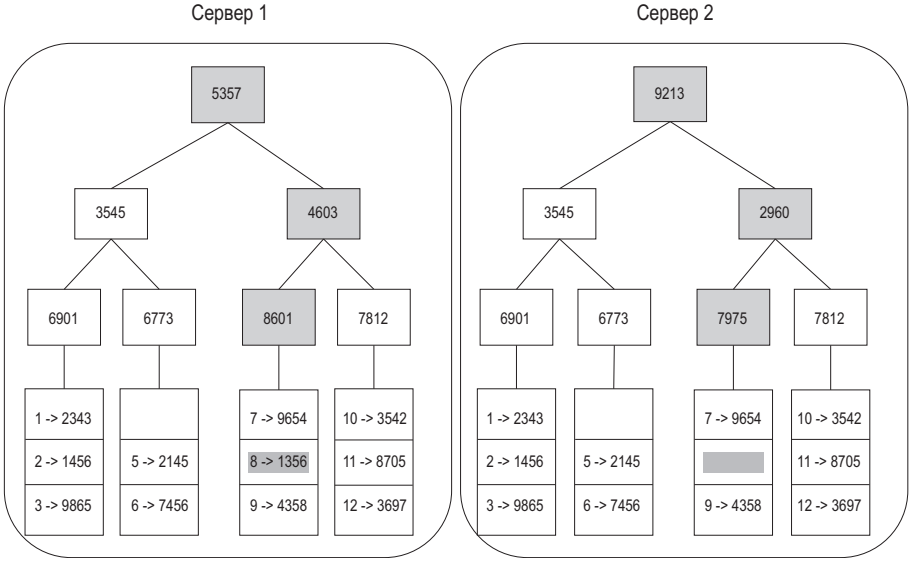


Рис. 6.16

При использовании деревьев Меркла количество данных, которые нужно синхронизировать, прямо пропорционально отличиям между двумя репликами и не зависит от того, сколько всего данных в них хранится. В реальных системах бакеты довольно большие. Например, на один миллион бакетов может приходиться по одному миллиарду ключей — то есть 1000 ключей в каждом бакете.

Обработка неполадок уровня ЦОД

Неполадки уровня ЦОД могут быть вызваны отключениями электричества, разрывами сети, стихийными бедствиями и т. д. Чтобы создать систему, способную с ними справиться, данные необходимо реплицировать по нескольким ЦОД. Даже если один ЦОД станет полностью недоступным, пользователи по-прежнему смогут получить данные из других центров обработки.

Диаграмма архитектуры системы

Итак, мы обсудили разные технические аспекты проектирования хранилища типа «ключ–значение». Теперь можно перейти к диаграмме архитектуры, показанной на рис. 6.17.

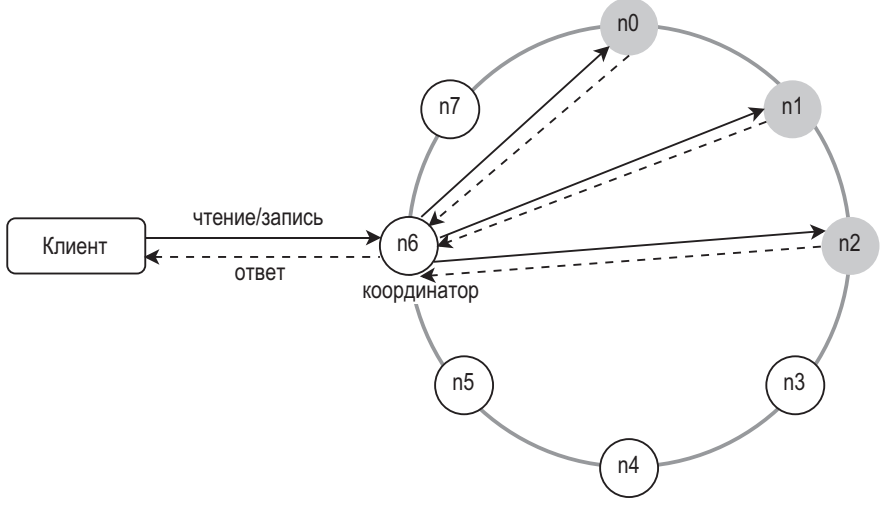


Рис. 6.17

Ниже перечислены основные особенности этой архитектуры.

- Клиенты взаимодействуют с хранилищем типа «ключ–значение» через простые API: get(ключ) и put(ключ, значение).
- Координатор — это узел, который выступает прокси-сервером между клиентом и хранилищем.

- Узлы распределяются по кольцу с использованием согласованного хеширования.
- Система полностью децентрализована, поэтому добавление и удаление узлов можно проводить автоматически.
- Данные реплицируются по разным узлам.
- Нет единой точки отказа, так как у каждого узла один и тот же набор обязанностей.

Поскольку архитектура децентрализована, каждый узел выполняет множество задач (рис. 6.18).



Рис. 6.18

Маршрут записи

На рис. 6.19 показано, что происходит, когда запрос на запись направляется к определенному узлу. Пожалуйста, обратите внимание на то, что предложенная архитектура маршрутов записи/чтения во многом позаимствована у Cassandra [8].

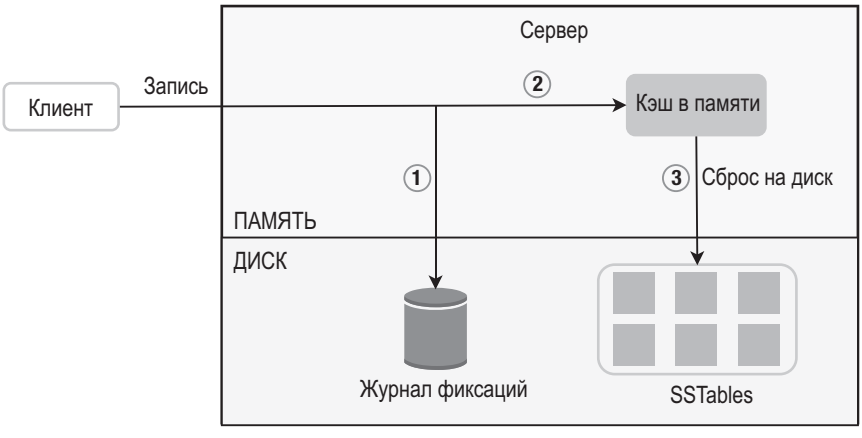


Рис. 6.19

1. Запрос на запись сохраняется в файл с логом коммитов.
2. Данные записываются в кэш, размещенный в памяти.
3. Когда кэш полностью заполняется или достигает определенного лимита, данные сбрасываются на диск в SSTable [9]. *Примечание*: SSTable (sorted-string table — «таблица с сортированием строк») — это упорядоченный список пар <ключ, значение>. Если вы хотите узнать больше об SStable, обратитесь к справочному материалу [9].

Маршрут чтения

Когда запрос на чтение направляется к определенному узлу, система сначала проверяет, находятся ли требуемые данные в кэше. Если это так, система возвращает их клиенту, как показано на рис. 6.20.

Если в памяти данных нет, они берутся с диска. Нам нужен эффективный способ поиска таблицы SSTable, в которой содержится ключ. Для этого часто используется фильтр Блума [10].

На рис. 6.21 показан маршрут чтения, когда данных нет в памяти.

1. Сначала система проверяет, есть ли данные в памяти, и если это не так, выполняется второй пункт.

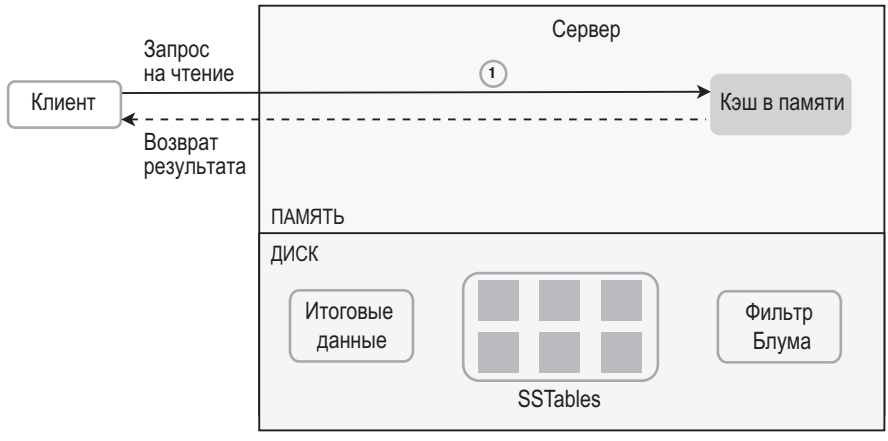


Рис. 6.20

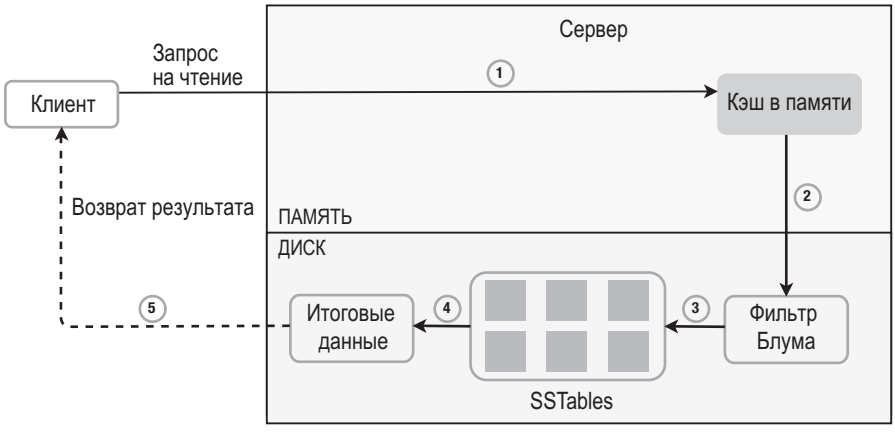


Рис. 6.21

2. Система проверяет фильтр Блума.
3. Фильтр Блума позволяет определить, в какой из таблиц SSTable может находиться ключ.
4. SSTable возвращает итоговый набор данных.
5. Итоговый набор данных возвращается клиенту.

ИТОГИ

В этой главе было рассмотрено множество концепций и техник. Чтобы вы точно все запомнили, ознакомьтесь с таблицей ниже, где даны характеристики распределенного хранилища типа «ключ–значение» и способы их реализации.

Таблица 6.2

Цель/задача	Методика
Возможность хранить объемные данные	Использование согласованного хеширования для распределения нагрузки между серверами
Высокая доступность при чтении	Репликация данных Конфигурация с несколькими ЦОД
Высокая доступность при записи	Версионирование и разрешение конфликтов с использованием векторных часов
Секционирование наборов данных	Согласованное хеширование
Инкрементальная масштабируемость	Согласованное хеширование
Гетерогенность	Согласованное хеширование
Регулируемая согласованность	Консенсус кворума
Обработка временных сбоев	Нестрогий кворум и прозрачная передача
Обработка бессрочных сбоев	Дерево Меркла
Обработка сбоев уровня ЦОД	Репликация между разными ЦОД

СПРАВОЧНЫЕ МАТЕРИАЛЫ

[1] Amazon DynamoDB: https://aws.amazon.com/dynamodb/

[2] memcached: https://memcached.org/

[3] Redis: https://redis.io/

[4] Dynamo: Amazon's Highly Available Key-value Store: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf

[5] Cassandra: https://cassandra.apache.org/

[6] Bigtable: A Distributed Storage System for Structured Data: https://static.googleusercontent.com/media/research.google.com/en// archive/bigtable-osdi06.pdf

[7] Дерево Меркла: https://ru.wikipedia.org/wiki/Дерево_хешей

[8] Cassandra architecture: https://cassandra.apache.org/doc/latest/architecture/

[9] SStable: https://www.igvita.com/2012/02/06/sstable-and-log-structured-storage-leveldb/

[10] Фильтр Блума: https://ru.wikipedia.org/wiki/Фильтр_Блума

7

ПРОЕКТИРОВАНИЕ ГЕНЕРАТОРА УНИКАЛЬНЫХ ИДЕНТИФИКАТОРОВ В РАСПРЕДЕЛЕННЫХ СИСТЕМАХ

В этой главе вам предлагается спроектировать генератор уникальных идентификаторов в распределенной системе. Возможно, использование первичного ключа с атрибутом `auto_increment` в традиционной базе данных станет первым, что придет вам в голову. Однако этот подход не работает в распределенных окружениях, так как отдельно взятый сервер БД слишком мал. К тому же генерировать уникальные ID в рамках нескольких БД с минимальными задержками не так уж просто.

Вот несколько примеров уникальных идентификаторов:

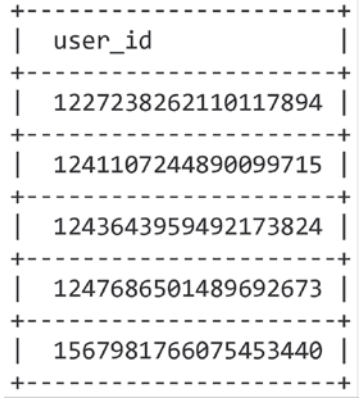


Рис. 7.1